3 Chapter 3 Project Management

Chapter 3 summarizes the work performed during the development of the project and discusses how the initial concept was developed into a real system. It focuses on the management and execution of the project through activities such as planning, feasibility evaluation, architectural selection, diagram design, implementation strategy, and testing preparation. The objective of this chapter is to illustrate that the project was not merely an idea supported by theory, but also a structured development effort carried out utilizing known software engineering and project management methods.

This chapter also demonstrates how the project developed from requirement identification to actual system realization. It covers the selected project management approach, defines the feasibility of the solution from many viewpoints, and explains the reasons behind the chosen architecture, tools, and implementation decisions. In doing so, the chapter gives proof that the project was executed in a controlled and deliberate way, with clear attention to practicality, risk management, and alignment between academic objectives and implemented outcomes.

3.1 Approach

Project management is a systematic framework for planning, coordinating, and regulating resources to attain specified project objectives within defined limitations. These include time, cost, and quality. In software development projects; proficient project management is crucial due to the intricacy of system requirements, interdependencies among components, and the necessity to mitigate risks throughout the development lifecycle.

In healthcare software systems, factors such as data security, ethical compliance, reliability, and system usability enhance the necessity for implementing suitable project management approaches. Consequently, the implementation of appropriate project management theories facilitates the methodical planning, monitoring, and execution of development activities in a controlled manner.

3.1.1 Selected Project Management Theory

Based on a detailed examination of project management theories, Agile Project Management was selected as the most acceptable approach for the development of the proposed Health Monitoring and Doctor Channeling system.

Agile project management is based on iterative development, adaptability, and continual improvement, making it well suited for software systems whose needs are not totally static at the outset. The suggested system comprises several interactive and data-driven components such as patient health management, health document upload and categorization, document upload with metadata organization and structured health record management, category-based symptom prediction using interpretable machine learning (logistic regression), GP appointment workflows with status tracking and notifications, administrative governance, and email-based notifications. These components require progressive refining as system understanding, usability considerations, and integration demands evolve during development.

A fundamental argument for selecting Agile is its ability to facilitate incremental release of functionality. Instead of aiming to design and build the entire system in a single development cycle, Agile allows the system to be created through smaller, manageable iterations. This technique lowers complexity, enhances visibility of progress, and facilitates early discovery of design or implementation difficulties. For a healthcare-related system where clarity, usability, and ethical bounds are crucial, this incremental method assures that system features may be tested and adjusted regularly.

Furthermore, Agile provides flexible requirement management, which is necessary for this project. While the overall system scope and boundaries were clearly established, specific elements such as user interface layouts, presentation formats of uploaded documents and symptom insights, and the structure and presentation of category-level predictive outputs require refining as the system evolved. Agile allows such refinements to be included without affecting the overall project plan, ensuring alignment with both functional objectives and non-functional criteria.

Another major reason for picking Agile is its emphasis on continuous quality assurance. By combining design, implementation, and testing within each iteration, Agile decreases the likelihood of late-stage problems and promotes early verification of system behavior. This is particularly critical for features requiring sensitive health data and machine learning-assisted

advisory outputs, where errors or ambiguities could impact user safety, and system reliability. Overall, Agile project management was picked because it provides a realistic, adaptable, and organized framework that matches with the scale, complexity, and ethical duties of the proposed system.

3.1.2 Application of the Selected Project Management Theory

The Agile project management approach will be followed throughout the project by structuring development into iterative cycles, each with clearly defined objectives, deliverables, and review points. This ensures that development continues in a regulated yet adaptable manner.

1. Initial Planning and Backlog Formation

At the start of the project, high-level system objectives, scope boundaries, and limitations were created. Functional and non-functional needs were defined and translated into a prioritized list of development tasks. These activities generated a development overview that influenced iteration planning. Core system operations with strong dependency relationships such as user authentication, role-based access control, and data storage, will be prioritized early to create a stable basis for subsequent improvements.

2. Iteration-Based Development

Early iterations will focus on core components such as patient registration, profile management, document upload with metadata, symptom history tracking, category-based symptom prediction using logistic regression, GP appointment booking workflows, inquiry communication features, and notification delivery for confirmations, updates and reminders Within each iteration, activities follow a structured sequence:

- Clarification of functional objectives for the iteration
- Design of interfaces, workflows, and data structures
- Implementation of selected features
- Verification through testing and review
- Refinement based on detected gaps or improvements

This iterative framework guarantees that functionality is delivered progressively and remains consistent with project objectives.

3. Continuous Review and Refinement

At the completion of each cycle, implemented features will be assessed against the established criteria and expected system behavior. This review process allows design tweaks, usability advancements, and performance optimizations to be integrated into future revisions. In particular, predictive features will be examined to verify that results were category-based, nondiagnostic, clearly explained, and consistent with the ethical boundaries stated in the project scope.

4. Quality Assurance Embedded in Agile Cycles

Quality assurance operations will be included within each iteration rather than considered as a separate final phase. Individual components were verified by unit testing, while interconnections between modules were reviewed through integration testing. End-to-end workflows like a patient submitting symptoms, uploading documents, submitting symptoms, uploading documents, reviewing symptom history, requesting a GP appointment, sending inquiries, and receiving notifications will be examined to ensure consistency and reliability across the system.

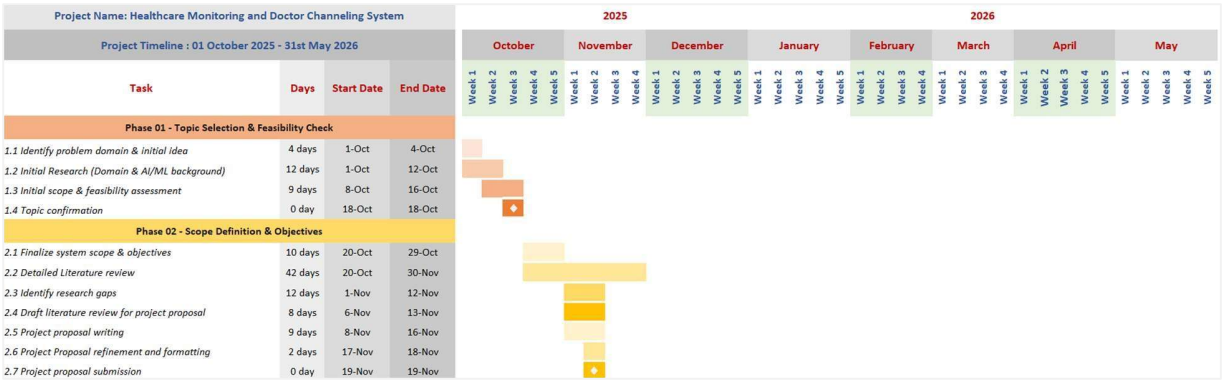
5. Documentation and Progress Monitoring

Although Agile stresses adaptation, detailed documentation will be maintained throughout the project. Design decisions, interface architecture, and functional behaviors will be documented sequentially to maintain traceability and clarity. Progress will be evaluated at the conclusion of each cycle by assessing finished features against anticipated objectives, allowing prompt identification of delays or modifications.

6. Managing Risks and Constraints

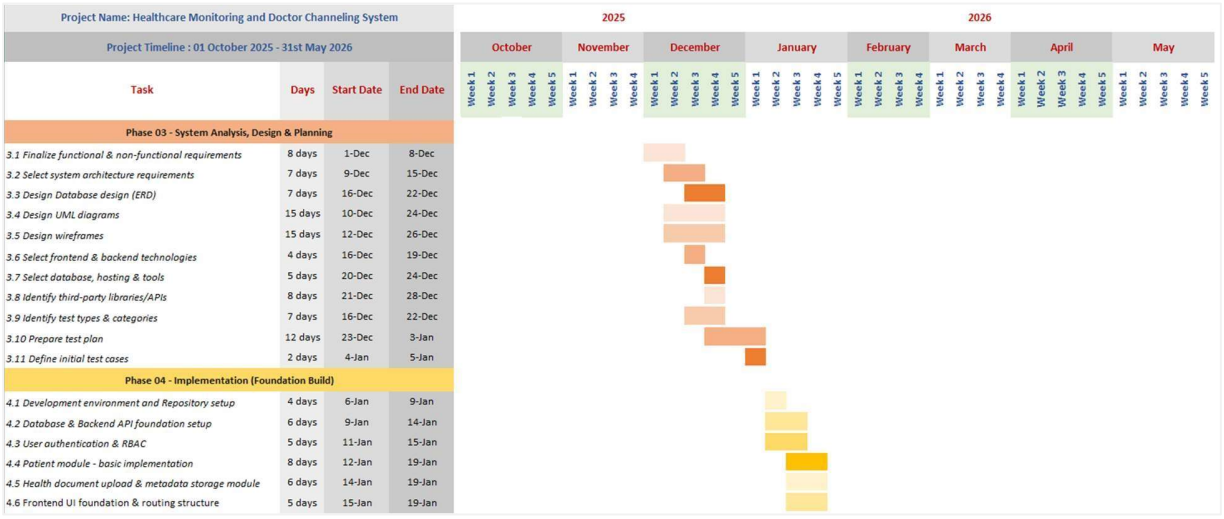
Agile encourages early identification and reduction of technical and design risks. Potential concerns relating to data security, system performance, and AI advisory outputs interpretation will be addressed sequentially through rigorous design, testing, and refining. This technique reduces the risk of severe complications appearing at later stages of development. Feasibility is maintained by limiting consultations to General Physicians, and having a category-based symptom checker result.

3.2 Initial Project Plan



Project Name: Healthcare Monitoring and Doctor Channeling System			
Project Timeline : 01 October 2025 - 31st May 2026			
Task	**Days**	**Start Date**	**End Date**
Phase 01 - Topic Selection & Feasibility Check			
1.1 Identify problem domain & initial idea	4 days	1-Oct	4-Oct
1.2 Initial Research (Domain & AI/ML background)	12 days	1-Oct	12-Oct
1.3 Initial scope & feasibility assessment	9 days	8-Oct	16-Oct
1.4 Topic confirmation	0 day	18-Oct	18-Oct
Phase 02 - Scope Definition & Objectives			
2.1 Finalize system scope & objectives	10 days	20-Oct	29-Oct
2.2 Detailed Literature review	42 days	20-Oct	30-Nov
2.3 Identify research gaps	12 days	1-Nov	12-Nov
2.4 Draft literature review for project proposal	8 days	6-Nov	13-Nov
2.5 Project proposal writing	9 days	8-Nov	16-Nov
2.6 Project Proposal refinement and formatting	2 days	17-Nov	18-Nov
2.7 Project proposal submission	0 day	19-Nov	19-Nov

Figure 4: Gantt Chart - Phase 1 & 2



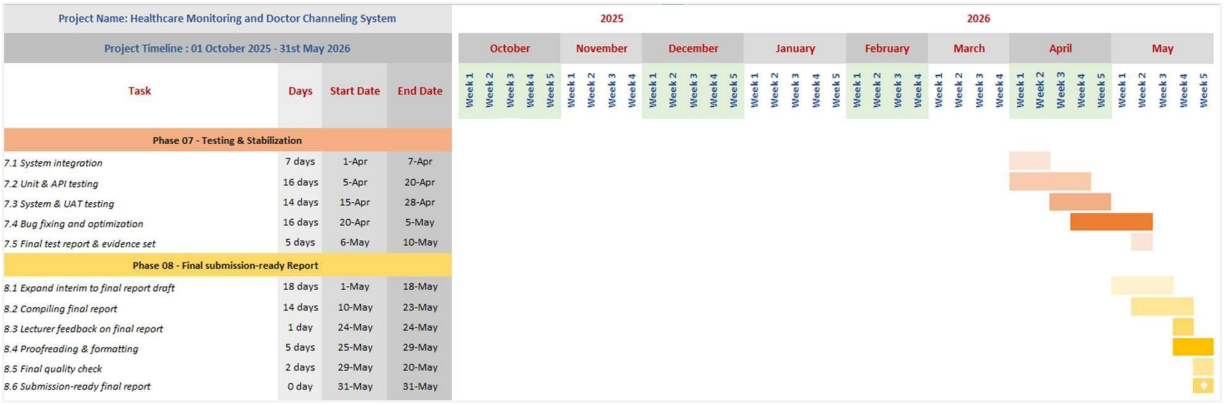
Project Name: Healthcare Monitoring and Doctor Channeling System			
Project Timeline : 01 October 2025 - 31st May 2026			
Task	**Days**	**Start Date**	**End Date**
Phase 03 - System Analysis, Design & Planning			
3.1 Finalize functional & non-functional requirements	8 days	1-Dec	8-Dec
3.2 Select system architecture requirements	7 days	9-Dec	15-Dec
3.3 Design Database design (ERD)	7 days	16-Dec	22-Dec
3.4 Design UML diagrams	15 days	10-Dec	24-Dec
3.5 Design wireframes	15 days	12-Dec	26-Dec
3.6 Select frontend & backend technologies	4 days	16-Dec	19-Dec
3.7 Select database, hosting & tools	5 days	20-Dec	24-Dec
3.8 Identify third-party libraries/APIs	8 days	21-Dec	28-Dec
3.9 Identify test types & categories	7 days	16-Dec	22-Dec
3.10 Prepare test plan	12 days	23-Dec	3-Jan
3.11 Define initial test cases	2 days	4-Jan	5-Jan
Phase 04 - Implementation (Foundation Build)			
4.1 Development environment and Repository setup	4 days	6-Jan	9-Jan
4.2 Database & Backend API foundation setup	6 days	9-Jan	14-Jan
4.3 User authentication & RBAC	5 days	11-Jan	15-Jan
4.4 Patient module - basic implementation	8 days	12-Jan	19-Jan
4.5 Health document upload & metadata storage module	6 days	14-Jan	19-Jan
4.6 Frontend UI foundation & routing structure	5 days	15-Jan	19-Jan

Figure 5: Gantt Chart - Phase 3 & 4



Project Name: Healthcare Monitoring and Doctor Channeling System			
Project Timeline : 01 October 2025 - 31st May 2026			
Task	**Days**	**Start Date**	**End Date**
Phase 05 - Interim Report			
5.1 Update literature review & research gap	5 days	10-Jan	14-Jan
5.2 Compile Interim report	10 days	10-Jan	19-Jan
5.3 Lecturer feedback on interim report	1 day	19-Jan	19-Jan
5.4 Apply feedback & finalize interim	2 days	19-Jan	20-Jan
5.5 Interim report submission	0 day	20-Jan	20-Jan
Phase 06 - Full Implementation & Refinement			
6.1 Patient module - Extended Implementation	15 days	21-Jan	4-Feb
6.2 Manual lab value entry & History tables	12 days	5-Feb	16-Feb
6.3 Data visualization (Charts & Trend views)	10 days	13-Feb	22-Feb
6.4 Doctor module- basic implementation	15 days	5-Feb	20-Feb
6.5 Appointment booking & receipt workflow	22 days	21-Feb	14-Mar
6.6 Notification system - basic implementation	8 days	9-Mar	16-Mar
6.7 AI/ML model development & integration	18 days	27-Feb	16-Mar
6.8 Doctor & Admin dashboards	16 days	15-Mar	31-Mar

Figure 6: Gantt Chart - Phase 5 & 6.



Project Name: Healthcare Monitoring and Doctor Channeling System			
Project Timeline : 01 October 2025 - 31st May 2026			
Task	Days	Start Date	End Date
Phase 07 - Testing & Stabilization			
7.1 System integration	7 days	1-Apr	7-Apr
7.2 Unit & API testing	16 days	5-Apr	20-Apr
7.3 System & UAT testing	14 days	15-Apr	28-Apr
7.4 Bug fixing and optimization	16 days	20-Apr	5-May
7.5 Final test report & evidence set	5 days	6-May	10-May
Phase 08 - Final submission-ready Report			
8.1 Expand interim to final report draft	18 days	1-May	18-May
8.2 Compiling final report	14 days	10-May	23-May
8.3 Lecturer feedback on final report	1 day	24-May	24-May
8.4 Proofreading & formatting	5 days	25-May	29-May
8.5 Final quality check	2 days	29-May	20-May
8.6 Submission-ready final report	0 day	31-May	31-May

Figure 7: Gantt Chart - Phase 7 & 8.

The project is planned by utilizing an organized Gantt chart., in order to have a clear visibility of tasks, dependencies, and milestone deliverables across the whole lifecycle. The project duration is about 8 months (01 October 2025 and 31 May 2026) which includes topic selection, literature study, system analysis and design, implementation, testing, and final report completion. To guarantee that the work is arranged rationally and stays in line with the standards for academic submission, the plan is divided into eight phases.

The Gantt chart, which displays scheduled task durations, start and end dates, and the distribution of activities along the timeline, is shown in Figures 4,5,6 and 7. In order to facilitate progress monitoring and feedback-driven improvement, the chart additionally highlights important milestone points (topic confirmation, proposal submission, interim submission, and final submission) and shows overlapping tasks where parallel progress is possible without disrupting dependencies.

Gantt Chart Summary:

The Gantt chart includes:

- task names together with estimated durations and date ranges;
- progression across weeks and months;
- controlled overlap of tasks where dependencies allow;
- milestone markers that correspond to academic deliverables (indicated by a diamond)

Task Clarification (where titles are non-specific)

Task	Explanation
Identify third-party libraries/APIs	Selection of charting, notification, and utility libraries required for frontend and backend
AL/ML model development and integration	Training and integration of Logistic Regression model for category-based symptom prediction and probability outputs.
Doctor and Admin dashboards	Implementation of authorized analytics dashboards for doctors and administrators using role-restricted datasets.
Receipt verification workflow	Administrative review process for uploaded payment receipts, including approve/rejected decisions and reason capture.
System and UAT testing	Execution of integration testing and user acceptance testing to validate end-to-end workflows and resolve defects.

Table 2: Task classification of non-specific task names in the Gantt chart.

Planning Justification:

The plan follows a phased lifecycle to ensure that academic deliverables and technical development are completed realistically within the available timeframe. Early phases prioritize feasibility, literature grounding, and design specification so that requirements and architecture are validated before major implementation begins, reducing the risk of rework and uncontrolled scope changes

Limited overlap between design, implementation, and documentation tasks is included to improve time efficiency while maintaining dependency control. Foundation development begins immediately after core design artefacts to enable incremental progress and early technical validation. Major submission points act as checkpoints for review and refinement, and dedicated

time reserved for integration, stabilization, and final documentation to ensure the delivered system is reliable and the final report is consistent with implemented outcomes.

3.3 Problems and Challenges to the Implementation Plan

Several obstacles were discovered throughout the implementation of the Health Monitoring and Doctor Channeling System, which prompted revisions to the initial Gantt chart and project plan. While the first timetable was structured sequentially, real-world development restrictions, evolving needs, and integration challenges made it necessary to adopt a more flexible and iterative approach.

To tackle these issues efficiently, development tasks were rearranged, parallel execution was introduced, and specific components were polished many times to assure system dependability and usability. The following are the significant issues faced during the project, along with the adjustments made and the rationale behind them.

1. Difficulty in Integrating Power BI for Administrative Analytics

One of the primary issues addressed was the integration of Power BI dashboards into the system for administrative analytics. While Power BI provided powerful visualization capabilities, incorporating it within the online application and establishing secure access control was more complicated than initially thought. Issues such as authentication difficulties, embedding restrictions, and connectivity challenges created delays during installation.

To solve this, more time was given for configuring Power BI service integration and testing embedded reports. Alternative methodologies were also investigated before finalizing a stable integration mechanism.

This difficulty emerged because external analytics tools work differently from core system components, and maintaining compatibility with the backend (ASP.NET) and frontend (React) needed additional configuration. Despite delays, this connection considerably improved the system’s capacity to convey insights in a structured and user-friendly manner.

2. Frequent Database Schema Modifications During Development

The initial database architecture did not properly capture all system interactions, especially as additional features such as inquiry management, appointment procedures, and role-based access evolved during deployment. As a result, the database structure had to be adjusted many times, including adding new tables, modifying relationships, and changing attributes.

These improvements needed periodic database migrations using SQL Server (SSMS), along with adjustments to backend models and API endpoints. In certain cases, previously established features have to be rewritten to fit with new structures.

This was important because system requirements became obvious only during development. Iterative improvement of the database ensured greater data organization, scalability, and consistency, even though it momentarily affected the development timeline.

3. Challenges in Synchronizing Frontend and Backend Development

The system was constructed using React for the frontend and ASP.NET Core Web API for the backend. Synchronizing these two levels posed issues, particularly when API endpoints were adjusted or when data structures changed.

At various places, inconsistencies between frontend expectations and backend responses generated issues such as improper data presentation, broken forms, and unsuccessful API requests. Debugging these difficulties needed coordination between frontend and backend logic.

To remedy this, development was altered to adopt a more iterative integration method, where APIs were tested alongside UI components rather than after full completion.

This task shows the complexity of full-stack development, where changes in one layer directly affect another. The solution increased system stability and ensured smoother integration.

4. Scope Adjustments During Development

Although the initial system design covered essential health monitoring and appointment functions, new functionalities such as the inquiry module (patient/doctor communication with admin) and improvements to user workflows were introduced throughout development. These additions increased system complexity and necessitated improvements to both frontend

interfaces and backend logic. As a result, certain projected timetables had to be modified, and development jobs were overlapped to compensate.
The scope expansion was important to improve system usability and integrate the program with real-world healthcare system needs. While it produced departures from the original Gantt chart, it considerably boosted the overall value of the system.

5. System Integration and Testing Complexity

During integration, various challenges developed while mixing diverse system components, including authentication, appointment management, AI prediction, and notification services. Data discrepancies and unusual behavior were detected when modules interacted with each other. To solve this, additional time was allotted for integration testing and debugging. Testing was also undertaken in parallel with development to discover difficulties early.
This problem was foreseen in a system with several interconnected units. Resolving these issues guaranteed that all components performed cohesively and increased overall system reliability.

6. AI Model Integration and Output Handling

Integrating the Logistic Regression model into the system brought issues related to data processing, input formatting, and output interpretation. Ensuring that symptom inputs from users were accurately digested and translated into meaningful predictions needed many modifications. Additionally, effort needs to be made to convey AI outputs in a safe and clear manner, avoiding misinterpretation by consumers.
To remedy this, the model integration process was modified, and unambiguous advisory signals and probability-based outputs were implemented.
This ensured that the AI function remained supportive rather than deceptive, complying with ethical system design standards.

7. Time Constraints and Deviation from Original Gantt Chart

One of the significant obstacles was the inability to strictly follow the original Gantt chart due to the cumulative impact of technical issues, scope modifications, and integration complications. As a result:

- Task durations had to be reduced

- Multiple jobs were conducted in concurrently
- Testing and documentation were conducted alongside development

This requires adopting a more flexible and adaptive scheduling technique rather than a strictly sequential one.

Despite these deviations, the project was successfully completed within the amended timetable by good time management and prioritization of essential features.

8. Development Environment and Tool Coordination

Using multiple tools such as Visual Studio (backend), Visual Studio Code (frontend), SQL Server Management Studio (database), and Power BI posed coordination issues. Managing dependencies, customizations, and compatibility amongst various tools took considerable effort. Version mismatches, environment setup difficulties, and deployment inconsistencies occasionally slowed down development.

These problems were addressed by uniform environment configuration, testing, and tool alignment.

Conclusion

The development of the Health Monitoring and Doctor Channeling System faced various practical obstacles relating to integration, database architecture, scope evolution, and time management. These issues made it difficult to precisely stick to the original Gantt plan. However, by adopting an iterative development methodology, incorporating parallel task execution, and refining system components incrementally, these issues were efficiently controlled. The improvements made during development not only corrected technical concerns but also improved the overall quality, usefulness, and reliability of the system.

3.4 Final Project Record

3.4.1 Deviations and Adjustments from the Original Plan

Although the original Gantt chart presented a well-structured and chronological plan for project execution, various practical issues found throughout development required considerable revisions to the timetable. As a result, the final Gantt chart reflects a more realistic and optimized representation of the project timeline.

1. Reduction in Overall Project Duration

The original project plan extended until 31 May 2026, however the final plan required submission by 19 April 2026. This resulted in a decrease of around 6 weeks, necessitating compression of implementation, testing, and documentation phases.

2. Phases that Remained Largely Unchanged

The following phases were accomplished closely according to the original plan:

- Phase 01 - Topic Selection & Feasibility
- Phase 02 - Scope Definition & Literature Review
- Phase 03 - System Analysis & Design
- Phase 04 - Foundation Implementation
- Phase 05 - Interim Report

These early years were crucial for developing a strong academic and technical foundation. Since they entailed structured research and planning efforts, they followed a generally consistent timeframe with minimal variance.

3. Major Changes in Implementation Phase (Phase 06)

The initial design allotted a longer length for full system implementation with a more progressive approach. However, due to time restrictions and evolving system requirements:

- Task durations were lowered
- Multiple modules were created in parallel (patient module, doctor module, AI integration, and appointment system)

- Additional features such as the inquiry module were included
- Database schema modifications required revision and multiple migrations

These adjustments resulted in a move from a linear development strategy to an iterative and concurrent development model.

4. Advancement of Testing Phase (Phase 07)

In the original Gantt chart, testing was scheduled as a separate phase after full implementation. However, in the final plan:

- Testing began early and coincided with development
- Unit testing, API testing, and system testing were completed in parallel
- Bug fixes and optimization occurs continuously instead than at the conclusion This adjustment helped discover issues earlier and lowered the likelihood of serious defects during final phases.

5. Early Start of Documentation and Final Report

Originally, final report preparation was intended mostly after system completion. However, due to the rushed schedule:

- Report writing began during implementation
- Screenshots, diagrams, and documentation were generated alongside development Final report compilation overlapped with testing phase

This guaranteed that paperwork was completed on schedule without delaying submission.

6. Increased Use of Parallel Task Execution

One of the most major modifications was the shift from a sequential process to a parallel execution model:

- Development, testing, and documentation were completed simultaneously
- Multiple modules were implemented sequentially
- Integration and debugging were handled constantly

This adjustment was essential to compensate for delays and fulfill the earlier submission date.

7. Delays and Their Impact

Several delays contributed to departures from the original Gantt chart:

- Integration issues (Power BI, AI model, frontend-backend communication)
- Frequent database schema modifications and migrations
- Scope modifications such as adding the inquiry module
- Debugging and integration concerns between system components

These delays rendered it impractical to precisely follow the planned timeframe and needed restructuring of later phases.

8. Tasks Completed Earlier than Planned

Some tasks were launched earlier than initially anticipated to optimize time usage:

- Testing activities started before full completion of installation
- Documentation duties were conducted alongside development
- Integration and bug fixing were completed continuously instead of being delayed

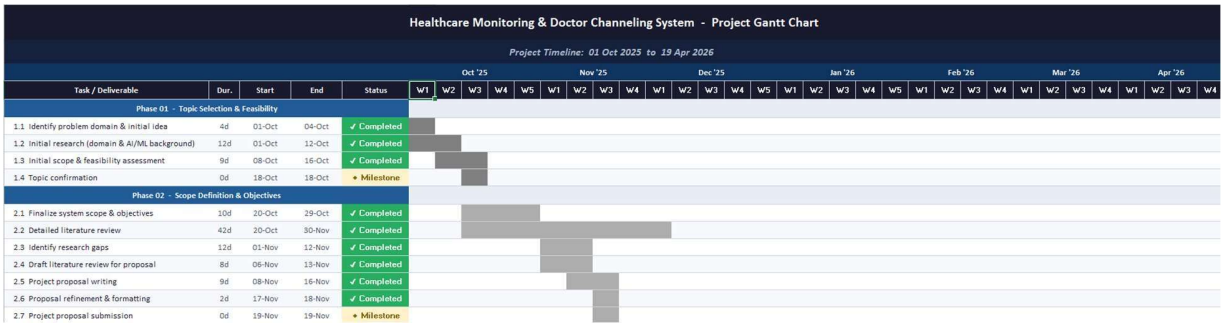
This proactive strategy helped recoup lost time and guaranteed seamless project progression.

Conclusion

The final Gantt chart reflects an efficient and realistic execution plan that evolved from the original schedule. While the initial design offered a strong foundation, real development issues needed revisions in task lengths, timing, and execution technique.

The shift towards parallel job execution, early testing, and ongoing documentation enabled the project to be completed successfully within the new timeframe. These adjustments not only addressed delays but also improved general efficiency, system stability, and alignment between implementation and documentation.

3.4.2 Final Revised Gannt Chart

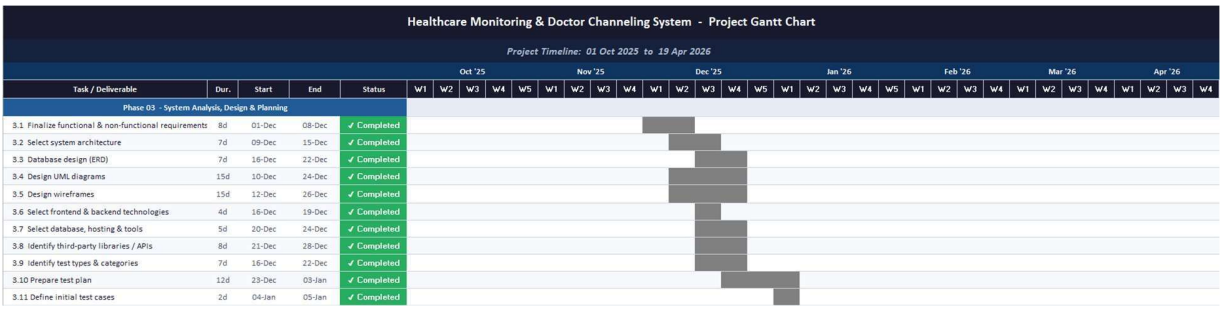


Healthcare Monitoring & Doctor Channeling System - Project Gantt Chart

Project Timeline: 01 Oct 2025 to 19 Apr 2026

Task / Deliverable	Dur.	Start	End	Status
Phase 01 - Topic Selection & Feasibility				
1.1 Identify problem domain & initial idea	4d	01-Oct	04-Oct	✓ Completed
1.2 Initial research (domain & AI/ML background)	12d	01-Oct	12-Oct	✓ Completed
1.3 Initial scope & feasibility assessment	9d	08-Oct	16-Oct	✓ Completed
1.4 Topic confirmation	0d	18-Oct	18-Oct	◆ Milestone
Phase 02 - Scope Definition & Objectives				
2.1 Finalize system scope & objectives	10d	20-Oct	29-Oct	✓ Completed
2.2 Detailed literature review	42d	20-Oct	30-Nov	✓ Completed
2.3 Identify research gaps	12d	01-Nov	12-Nov	✓ Completed
2.4 Draft literature review for proposal	8d	06-Nov	13-Nov	✓ Completed
2.5 Project proposal writing	9d	08-Nov	16-Nov	✓ Completed
2.6 Proposal refinement & formatting	2d	17-Nov	18-Nov	✓ Completed
2.7 Project proposal submission	0d	19-Nov	19-Nov	◆ Milestone

Figure 8: Final Gannt Chart - Phase 1 & 2

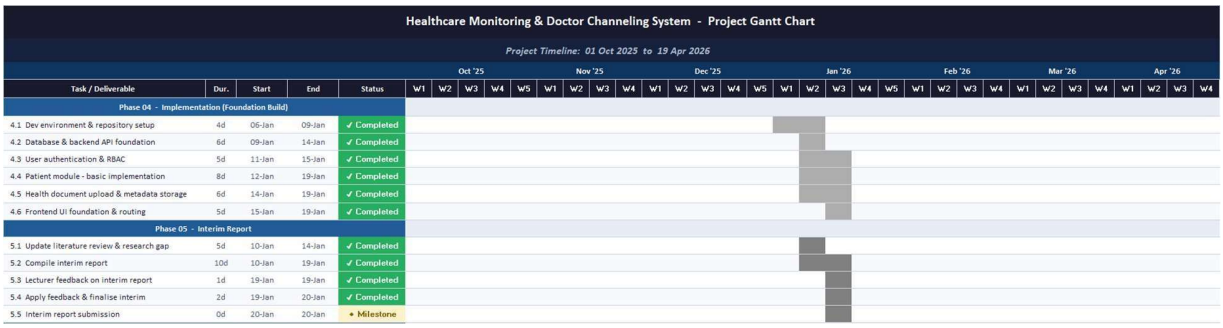


Healthcare Monitoring & Doctor Channeling System - Project Gantt Chart

Project Timeline: 01 Oct 2025 to 19 Apr 2026

Task / Deliverable	Dur.	Start	End	Status
Phase 03 - System Analysis, Design & Planning				
3.1 Finalize functional & non-functional requirements	8d	01-Dec	08-Dec	✓ Completed
3.2 Select system architecture	7d	09-Dec	15-Dec	✓ Completed
3.3 Database design (ERD)	7d	16-Dec	22-Dec	✓ Completed
3.4 Design UML diagrams	15d	10-Dec	24-Dec	✓ Completed
3.5 Design wireframes	15d	12-Dec	26-Dec	✓ Completed
3.6 Select frontend & backend technologies	4d	16-Dec	19-Dec	✓ Completed
3.7 Select database, hosting & tools	5d	20-Dec	24-Dec	✓ Completed
3.8 Identify third-party libraries / APIs	8d	21-Dec	28-Dec	✓ Completed
3.9 Identify test types & categories	7d	16-Dec	22-Dec	✓ Completed
3.10 Prepare test plan	12d	23-Dec	03-Jan	✓ Completed
3.11 Define initial test cases	2d	04-Jan	05-Jan	✓ Completed

Figure 9: Final Gannt Chart - Phase 3



Healthcare Monitoring & Doctor Channeling System - Project Gantt Chart

Project Timeline: 01 Oct 2025 to 19 Apr 2026

Task / Deliverable	Dur.	Start	End	Status
Phase 04 - Implementation (Foundation Build)				
4.1 Dev environment & repository setup	4d	06-Jan	09-Jan	✓ Completed
4.2 Database & backend API foundation	6d	09-Jan	14-Jan	✓ Completed
4.3 User authentication & RBAC	5d	11-Jan	15-Jan	✓ Completed
4.4 Patient module - basic implementation	8d	12-Jan	19-Jan	✓ Completed
4.5 Health document upload & metadata storage	6d	14-Jan	19-Jan	✓ Completed
4.6 Frontend UI foundation & routing	5d	15-Jan	19-Jan	✓ Completed
Phase 05 - Interim Report				
5.1 Update literature review & research gap	5d	10-Jan	14-Jan	✓ Completed
5.2 Compile interim report	10d	10-Jan	19-Jan	✓ Completed
5.3 Lecturer feedback on interim report	1d	19-Jan	19-Jan	✓ Completed
5.4 Apply feedback & finalise interim	2d	19-Jan	20-Jan	✓ Completed
5.5 Interim report submission	0d	20-Jan	20-Jan	◆ Milestone

Figure 10: Final Gannt Chart - Phase 4 & 5



Healthcare Monitoring & Doctor Channeling System - Project Gantt Chart

Project Timeline: 01 Oct 2025 to 19 Apr 2026

Task / Deliverable	Dur.	Start	End	Status
Phase 06 - Full Implementation & Refinement				
6.1 Patient module - extended implementation	10d	21-Jan	30-Jan	✓ Completed
6.2 Document upload & metadata	8d	25-Jan	02-Feb	✓ Completed
6.3 Symptom checker + AI integration	10d	28-Jan	06-Feb	✓ Completed
6.4 Doctor module - basic implementation	10d	01-Feb	10-Feb	✓ Completed
6.5 Appointment system	10d	05-Feb	14-Feb	✓ Completed
6.6 Notification system	6d	08-Feb	14-Feb	✓ Completed
6.7 Inquiry module	5d	10-Feb	15-Feb	✓ Completed
6.8 Admin dashboards	6d	12-Feb	18-Feb	✓ Completed
6.9 Integration + bug fixing	10d	18-Feb	01-Mar	✓ Completed
Phase 07 - Testing & Stabilisation				
7.1 System integration testing	5d	01-Mar	05-Mar	✓ Completed
7.2 Unit & API testing	7d	03-Mar	10-Mar	✓ Completed
7.3 System & UAT testing	8d	08-Mar	15-Mar	✓ Completed
7.4 Bug fixing & optimisation	10d	10-Mar	20-Mar	✓ Completed
7.5 Test evidence & report	5d	15-Mar	20-Mar	✓ Completed

Figure 11: Final Gannt Chart - Phase 6 & 7

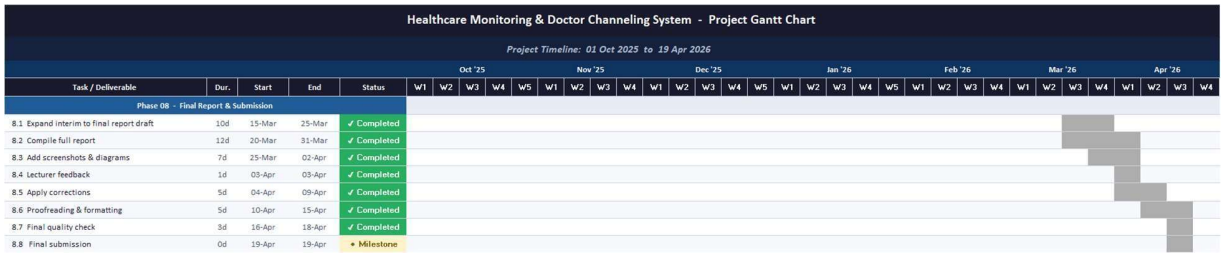


Figure 12: Final Gannt Chart- Phase 8

The project is executed utilizing an organized and efficient Gantt chart to guarantee clear visibility of tasks, dependencies, and milestone deliverables throughout the system development lifecycle. The updated project length spans roughly 6.5 months (01 October 2025 to 19 April 2026), covering topic selection, literature research, system analysis and design, implementation, testing, and final report completion. The new plan is structured into eight phases to preserve consistency with academic criteria while ensuring timely project delivery.

The improvised Gantt chart, displayed in Figures above, displays the updated scheduling of tasks, including altered durations, faster completion of development phases, and greater parallel execution of activities. Key milestone points such as subject confirmation, proposal submission, interim report submission, and final report submission are clearly noted. Compared to the initial plan, the amended chart indicates a more compressed timeframe with increased overlap between implementation, testing, and documentation work to achieve the advanced submission deadline.

Gantt Chart Summary (Final Plan) The
revised Gantt chart includes:

- job names with updated durations and altered date ranges;
- progression throughout weeks and months with constrained timelines;
- enhanced concurrent execution of jobs throughout implementation, testing, and reporting phases;
- milestone markers aligned with academic deliverables (represented by a diamond); □ early completion of development and testing phases prior to final submission. Execution of system-level integration testing and user acceptance testing with early parallel testing approach. |

Task Clarification (where titles are non-specific)

Task	Explanation
Identify third-party libraries/APIs	Selection and integration of required tools such as notification services and external analytics tools (e.g., Power BI embedding).
AL/ML model development and integration	Training and integration of Logistic Regression model for symptom prediction with advisory outputs.
Admin dashboards	Integration of analytics tools into the system and Implementation of operational and symptom-based dashboards.
Inquiry Module	Feature enabling patients and doctors to communicate with administrators for support and issue handling.
System and UAT testing	Execution of system-level integration testing and user acceptance testing with early parallel testing approach.

Table 3:: Task Clarification for non-specific tasks in the final Gannt chart

Planning Justification (Final Plan)

The redesigned strategy follows a staged yet flexible lifecycle to ensure that both technical implementation and academic deliverables are achieved within the decreased period. Early steps such as feasibility study, literature assessment, and system design remained essentially unchanged to provide a strong foundation for development.

In contrast to the initial concept, the implementation, testing, and documentation processes were optimized using parallel execution. Development of numerous modules (patient, doctor, AI, and

appointment systems) was carried out concurrently, while testing activities were began before full system completion. Additionally, report writing, diagram preparation, and documentation duties were started earlier and progressed alongside development.

This method saved idle time between phases and guaranteed that crucial milestones were completed without compromising system quality. The inclusion of overlapping tasks permitted efficient use of time while keeping control over dependencies, enabling successful completion of the project by the final submission deadline of April 19, 2026.

4 Chapter 4 Feasibility study

Chapter 4 offers the feasibility study of the planned system and assesses whether the idea was realistic, practical, and justifiable from operational, technological, economic, scheduling, ethical, and legal viewpoints. The goal of this chapter is to establish whether the system may be built effectively within the available resources, time frame, and technological scope while still staying useful, safe, and academically valid. In health-related systems, feasibility is especially crucial because the system must not only work successfully but also support secure handling of sensitive information and responsible use of digital decision-support functions.

Through this chapter, the research evaluates the preparedness of users to embrace the platform, the suitability of the technological stack, the affordability of the system, and the practicality of developing and completing the project within the provided timetable. It also addresses major ethical and legal considerations, particularly because the research contains personal health information and an AI-supported symptom checker. By examining all of these factors together, the chapter validates that the implemented solution is viable within the project scope and that it can operate in a manner that is secure, beneficial, managed, and ethically acceptable.

4.1 Operational Feasibility

The operational feasibility of the implemented Health Monitoring and Doctor Channeling System shows how well the system is currently accepted, used, and maintained in real-world health monitoring and consultation workflows. The platform is intended to connect seamlessly with existing practices in patient care, clinical consultation, and administrative management, resulting in minimal disturbance to daily operations while boosting coordination and efficiency.

- User Readiness and Role Clarity:

The system is built around well-defined user roles, such as patients, medical professionals, and system administrators, each with specific functions and access controls. These roles closely resemble real-world healthcare interactions, allowing users to comprehend and operate the system with ease. Patients execute standard tasks such as updating profiles, uploading health documents, managing structured document metadata, and scheduling appointments, while doctors adhere to

established clinical routines that include patient assessments and consultation documentation. This role clarity promotes smooth day-to-day operations and decreases resistance to system utilization.

- Integration with Existing Healthcare Workflows:

The system enhances rather than replaces established healthcare processes. Clinical consultations and follow-up care will continue as usual, with the system serving as a digital platform to organize data, display structured document histories, and improve communication. Medical practitioners retain complete control over clinical decisions, ensuring that the system serves as a useful tool rather than a disruptive operational shift.

- Usability and Accessibility for End Users:

Operational effectiveness is achieved through a simple, web-based interface that prioritizes navigation ease and information clarity. Structured document histories, symptom records, and appointment status indicators, together with guided booking processes, reduce complexity for both patients and clinicians. Standard web browsers provide cross-device usability without the need for specialist hardware or substantial technological knowledge.

- Support for Advisory, Non-Diagnostic AI Usage:

AI/ML-based features are used as advising tools to increase awareness rather than making decisions. Symptom forecasts and advice insights are clearly labeled as non-diagnostic and require validation by a skilled healthcare practitioner. This strategy reduces the danger of misuse while maintaining professional accountability in all healthcare choices.

- Administrative Oversight and Operational Control:

Administrators are given capabilities to manage user accounts, create and manage doctor accounts, track appointments, and monitor system activity. These controls guarantee that the platform adheres to stated policies and maintains accountability, security, and consistency throughout everyday operations.

- Continuity of Care and Communication Efficiency:

The system manages patient records, consultation histories, and sends appointment notices, updates, and reminders via system/email. Appointment confirmations and reminders help to reduce missed consultations and administrative costs, while also guaranteeing constant communication and continuity of care between patients and doctors. Notifications include the appointment type, date/time, and either an online meeting link or physical location information.

Potential Operational Challenges and Mitigations

Potential Challenge	Impact	Mitigation Strategy
Initial user hesitation toward digital workflow	Low-Medium	User-friendly interface and rolebased navigation
Misinterpretation of AI/ML generated insights	Medium	Clear advisory labels, probability display, and clinician validation guidance.
Incomplete or outdated patientuploaded data	Medium	Document categorization, timestamps, and clinician review
Increased administrative effort during early adoption	Low	General physician account provisioning by admin and automated notifications

Table 4: Potential Operational Challenges and Mitigations.

Conclusion:

The system is operationally feasible because it works well with existing healthcare processes, has clear user responsibilities, and prioritizes usability and professional monitoring. Minor operational issues may develop, but they are solvable through system design and administrative controls. Overall, the system performs well inside healthcare workflows, enhancing coordination, continuity of care, and service efficiency.

4.2 Technical Feasibility

The technical feasibility of the developed Health Monitoring and Doctor Channeling System has been proven by its successful deployment and operation utilizing readily available technology. The system runs on a web-based platform that uses common hardware, network infrastructure, and software technologies, allowing for creation, deployment, and maintenance without substantial technical constraints.

1. Hardware Requirements

a. Server Infrastructure:

The application backend and database are hosted in a cloud server environment. The server handles critical system activities such user authentication, health profile management, document storage, symptom submission, appointment booking, appointment status management, notification processing, and simple visualization support. Adequate processing capacity, memory, and secure storage allow for simultaneous user access and file handling. Backup systems and system dependability measures help to assure stability.

b. Client Devices:

Administrators and medical professionals use standard desktop or laptop computers to access system interfaces, check patient records, schedule appointments, and take notes on consultations. Patients use web browsers to access the system from their personal devices, such as computers, tablets, or smartphones. This removes the requirement for specialized hardware.

c. Peripheral Equipment:

Optional external devices such as printers can be used to generate physical records or summaries when necessary. While the system is mostly digital, modest printing capabilities offer flexibility in administrative or clinical scenarios.

2. Network Infrastructure

a. Internet Connectivity:
A stable and reliable internet connection was necessary to allow continuous access to the webbased platform. Internet connectivity offers remote access by patients and doctors, email notifications, and interaction with third-party services where required. Since the system does not rely on local-only networking, it stays available across multiple geographic regions.

b. Secure Communication:
HTTPS have been used to protect sensitive health information during communication between users and the server. This protects confidentiality and integrity of patient data during system usage.

c. Basic Networking Equipment:
Standard networking hardware such as routers and switches were suitable to support administrative access where applicable. No complex or specialized networking infrastructure is needed.

3. Software Requirements

a. Development Environment:
Development was carried out using ASP.NET Core Web API (C#) for backend services, React (SPA) for client interface, Microsoft SQL Server for persistence, and Python for Logistic Regression model for advisory symptom prediction, and PowerBi for analytics are integrated into the system through backend services. These technologies are widely adopted, well-documented, and suitable for iterative development and testing.

b. Database Management System:
A relational database management system was utilized to hold structured data such as user profiles, document metadata, consultation-related records, and appointment details. This assured data integrity, consistency, and dependable concurrent access.

c. Third-Party Service Integration:

The system interfaces with external services for appointment communication features, meeting link sharing for online consultations, and analytics utilizing well-established APIs. These connectors reduce development complexity and enable dependable communication operations.

d. Web Interface Framework:

React (SPA) with Bootstrap/React-Bootstrap is used to implement a responsive user interface across patient doctor, and administrator modules. This provides consistent user experience across devices.

4. Technical Support and Maintainability

a. Developer Familiarity:

The technologies selected for the system are widely used and well-documented, minimizing learning overhead and allowing rapid problem resolution during development.

b. Availability of Resources:

Required software platforms and infrastructure services are easily available, avoiding dependency-related delays and guaranteeing easy deployment.

c. Maintenance and Scalability:

The system is created using modular architecture concepts, allowing future additions, additional features, or performance optimizations without considerable redesign. Routine maintenance chores such as upgrades and backups can be conducted using conventional tools.

d. User Training and Documentation:

Clear documentation and simple user assistance promoted effective system utilization by all user groups, decreasing dependency on technical personnel for regular operations.

Potential Technical Challenges and Mitigation

Potential Challenge	Impact	Mitigation Strategy
Handling large health document uploads	Medium	File size validation and optimized storage handling
Dependence on third-party notification services	Medium	Use of reliable APIs and fallback notification mechanisms
Ensuring data security and privacy	High	Encryption, secure authentication and role-based access control
Performance under increased user load	Low-Medium	Scalable hosting and modular architecture.
ML model reliability and misinterpretation	Medium	Use advisory-only outputs, probability display, and clear disclaimers.

Table 5: Potential Technical Challenges and Mitigations.

Conclusion:

The proposed Health Monitoring and Doctor Channeling System is technically feasible, as it relies on commonly accessible hardware, solid network connectivity, and mature software technologies. While potential technical risks may exist, they are controllable through known design techniques and mitigation strategies. Overall, the system was developed, implemented, and maintained effectively within the given project scope without significant technological barriers.

4.3 Cost Feasibility

The cost feasibility of the established Health Monitoring and Doctor Channeling System assesses if the system was designed and is being maintained within reasonable financial limits. The system was deployed with publicly available technologies and infrastructure, eliminating the need for costly proprietary tools or specialized hardware.

- Initial Cost (One-time costs)

The initial expenditure was made during the development and deployment phases, which included system design, implementation, infrastructure setup, and integration with external services.

Cost Component	Estimated Cost (LKR)
System design and development	200,000
Server/ cloud environment setup	100,000
Hardware and peripherals (basic setup)	80,000
Software tools and utilities	30,000
Deployment and initial configuration	40,000
Third-party service setup	30,000
	TOTAL INITIAL COST: 480,000

Table 6: Initial Cost

- Ongoing Operational Costs (Annual)

The system currently incurs recurring costs for hosting, maintenance, and communication services.

Cost Component	Estimated Annual Cost (LKR)
Hosting and Infrastructure	90,000
Maintenance and technical support	120,000
Third-party services (Analytics, APIs)	40,000
Internet and communication costs	35,000
System upgrades and minor enhancements	45,000

TOTAL ANNUAL OPERATIONAL COST: 330,000

Table 7: Ongoing Operational Cost

- Cost Considerations:

The solution avoids costly infrastructure by relying on cloud-based hosting and open-source technology. No unique hardware is required, and maintenance tasks can be handled using regular development processes. The modular architecture ensures that future enhancements can be made without incurring considerable additional costs.

- Conclusion

The system is cost-effective because it was built with affordable technologies and continues to function at predictable and controlled expenses.

4.4 Economic Feasibility

The economic feasibility of the system determines if the advantages created by the system outweigh the entire cost over time. The solution now improves efficiency, lowers administrative expense, and promotes continuity of service, resulting in real operational advantages.

- Expected Annual Benefit

Benefit Category	Estimated Annual Benefit (LKR)
Improved workflow efficiency (appointment handling and reduced admin workload)	400,000
Reduced administrative effort through automation	150,000

Improved continuity of care and data organization	150,000
TOTAL EXPECTED ANNUAL BENEFITS: 700,000	

Table 8: Expected Annual Benefit

- Cost-Benefit Projection (5-year summary)

Year	Annual Cost (LKR)	Annual Benefit (LKR)	Net Benefit (LKR)	Cumulative Net Benefit (LKR)
0	520,000	0	-480,000	-480,000
1	330,000	700,000	370,000	-110,000
2	330,000	700,000	370,000	260,000
3	330,000	700,000	370,000	630,000
4	330,000	700,000	370,000	1000,000
5	330,000	700,000	370,000	1370,000

Table 9: Cost-Benefit Projection

Year 0 represents the one-time initial investment cost before operations begin.

- Payback Period

The payback period indicates the time required for the project to recover its initial investment through net annual benefits

$$Payback\ Period = \frac{Initial\ Investment}{Annual\ Net\ Benefit}$$

$$Payback\ Period = \frac{480{,}000}{370{,}000}$$

$$Payback\ Period \approx 1.30\ years$$

The Health Monitoring and Doctor Channeling system is expected to recover its initial investment in approximately 1 year and 4 months (≈ 1.30 $years$). This relatively short payback period demonstrates that the project will generate financial returns within an acceptable timeframe, supporting its economic feasibility.

 Return on Investment (ROI)

The Return on Investment (ROI) is used to evaluate the overall profitability of the project relative to the total cost incurred over the evaluation period.

$$Total\ cost\ over\ 5\ years = 520{,}000 + (330{,}000 \times 5)$$
$$Total\ cost\ over\ 5\ years = 2,\ 130{,}000\ LKR$$

$$ROI = \left(\frac{Cumulative\ Net\ Benefit}{Total\ Investment} \times 100\right)$$

$$ROI = \frac{1370{,}000}{2{,}130{,}000} \times 100$$

$$ROI \approx 64.3\%\ (36\%)$$

An ROI of approximately 36% indicates that the project delivers a strong financial return over the five-year period. This level of return suggests that the system is not only cost-recovering but also financially rewarding in the long term

Conclusion:
The system is economically feasible because it consistently delivers financial benefits that outweigh operating costs. The short payback period and strong return on investment demonstrate that the system is financially viable and profitable in the long run.

4.5 Schedule Feasibility

Schedule feasibility assesses if the Health Monitoring and Doctor Channeling System has been finalized within the established project timetable, ensuring adherence to the requisite standards of

development, testing, and documentation quality. The project was successfully completed within the scheduled timeframe from October 2025 to April 19, 2026, showcasing efficient time management and organized job execution.

The project timeline was meticulously crafted to equilibrate research, system design, implementation, testing, and report writing tasks. A systematic development strategy was employed, with explicit milestones and deliverables established for each phase. Core system functionalities were prioritized in the first implementation phases, while advanced additions and enhancements were executed concurrently to assure timely delivery.

- Key Project Milestones

Milestone	Planned Date
Topic Confirmation	18 October 2025
Project Proposal Submission	19 November 2025
Interim Report Submission	20 January 2026
Final System Completion & Testing	20 March 2026
Submission-ready Final Report	19 April 2026

Table 10: Key Project Milestones.

- Phase-based Schedule Overview

Project Phase	Planned Period
Topic Selection and Initial Research	October 2025
Scope Definition, Literature Review and Proposal	October – November 2025
System Analysis, Design and Planning	December 2025 – Early January 2026
Core Implementation (foundation build)	January 2026
Full Implementation and Feature Development	January 2026 – March 2026
Testing and System Stabilization	March 2026
Final Report Preparation	March 2026 – April 2026

Table 11: Phase-based Schedule overview.

- Use of Parallel Task Execution

To fulfill the extended submission date, the project employed a parallel execution technique throughout later phases. Development, testing, and documentation operations were carried out simultaneously to enhance time utilization.

Parallel Activities	Purpose
Literature review with proposal writing	Prevents delays and accelerates early progress
Implementation with interim report writing	Ensures availability of screenshots and technical evidence
Testing alongside implementation	Identifies issues early and improves stability
Report writing during development	Ensures documentation is completed on time
Diagram and screenshot preparation with system completion	Avoids last-minute documentation delays

Table 12: Use of Parallel Task Execution.

- Effort Distribution by Functional Area

Functional Area	Scope Summary	Feasibility Assessment
Core system modules	Authentication, patient profiles, document upload and metadata, symptom tracking	High

Appointment system	Appointment booking, status tracking, notifications	High
Doctor and administrator modules	Doctor scheduling, patient record review, inquiry handling, system monitoring, Operational and Symptom-based Analytics	High
AI-based features	Logistic Regression-based symptom prediction and advisory outputs	Medium
Integration and Bug fixing	System integration, error resolution, performance optimization	Medium
Testing and Validation	Unit testing, API testing, system testing, UAT validation	High
Final Documentation	Report writing, diagrams, screenshots, formatting, proofreading	High

Table 13: Effort Distribution by Functional Area

Schedule Optimization Strategy

To ensure completion before the final submission date, many optimizations were applied:

- Task durations were lowered without impacting essential functionality
- Multiple modules were created concurrently (e.g., patient module, symptom checker, and doctor module)
- Testing activities were commenced before full completion of all modules
- Documentation activities, including report writing and diagram creation, were initiated early and continued in parallel with development
- Final report preparation overlapped with testing and system stability phases

This strategy considerably reduced idle time between phases and enhanced overall efficiency.

Conclusion:

The project schedule is viable and has been successfully achieved within the stipulated timeframe. The use of phased development combined with parallel task execution provided efficient time management and secured timely completion of all system components. Despite the constrained timetable, all primary functionalities, testing activities, and documentation requirements were fulfilled by the final submission deadline of April 19, 2026. This demonstrates that the project timeframe was realistic, well-managed, and well implemented.

4.6 Ethical and Legal Feasibility

The Health Monitoring and Doctor Channeling System's ethical and legal viability is crucial since it processes sensitive personal and health-related information and incorporates AI-based decision-support. The system is built with clear ethical safeguards and legal knowledge to ensure responsible operation while also recognizing potential hazards and limitations.

1. Ethical Considerations

- Patient Data Privacy
  The system handles sensitive health information such as uploaded health documents, symptom reports, and consultation details. Access is regulated via role-based authorization, which ensures that only patients and authorized medical personnel have access to pertinent data.

- User Consent and Transparency
  Users explicitly consent to data collection, storage, and processing throughout system usage. Privacy notices and system explanations are prominently shown to encourage transparency and informed involvement.

- Risk of Misinterpretation
  AI-based symptom prediction results may not be comprehended by consumers. This risk is reduced by explicitly marking all outputs as advisory and non-diagnostic, as well as large disclaimers promoting expert medical advice.

- Clinical Accountability
  Qualified healthcare professionals are responsible for all clinical choices. The system does not make diagnosis or treatment recommendations, ensuring that medical accountability is maintained.

- Data Accuracy and Reliability
  Incomplete or outdated user-uploaded data might lead to inaccurate interpretations. To improve data dependability, the system allows for hierarchical document organization, timestamps, and symptom history tracking. Users and clinicians are encouraged to thoroughly evaluate records during consultations to guarantee accurate interpretation.

- Notification Safety
  System alerts and notifications offer appointment confirmations, updates, and reminders without revealing sensitive clinical information, minimizing privacy risks.

2. Legal Considerations

- Healthcare Data Protection
  Although the technology is not used in a regulated clinical setting, it conceptually conforms with healthcare data privacy standards like GDPR and HIPAA. These include confidentiality, secure data management, restricted access, and limited data exposure.

- Data Ownership and Usage
  Patient information is only utilized for system functionality and academic reasons. The system does not share, sell, or disseminate user data, minimizing the legal concerns connected with illicit use.

- System Liability Limitations
  The system does not offer medical diagnosis or medications. This considerably decreases legal liability for clinical decision-making. Clear disclaimers emphasize that the results are advisory and must be evaluated by healthcare specialists.

- Secure Data Storage and Transmission

  All data is securely communicated via HTTPS and saved with secure authentication and rolebased access control. To prevent illegal retrieval of sensitive documents, database access is controlled.

- Record Traceability and Auditability

  The system keeps basic audit logs for important actions including administrative updates, doctor account management, and appointment status changes. This promotes traceability and increases legal accountability.

Potential Ethical and Legal Challenges and Mitigations

Potential Challenge	Impact	Mitigation Strategy
Misuse of AI/ML outputs by patients	Medium	Advisory labelling, disclaimers, and clinician validation.
Unauthorized access to sensitive data	High	Role-based access control, secure authentication, and encryption
Incomplete or incorrect user-entered data and uploaded documents	Medium	Structured document management, timestamps, and clinician review
Future regulatory changes	Low-Medium	Modular design to support future policy updates and compliance improvement

Table 14:Potential Ethical and Legal Challenges and Mitigations

Conclusion:

The Health Monitoring and Doctor Channeling System is ethically and legally feasible since it includes data privacy protection, openness, professional monitoring, and appropriate AI use in its implementation. While there are ethical and legal concerns, particularly when it comes to sensitive data handling and AI interpretation, these risks can be efficiently addressed through secure system design, access control, permission processes, and defined advisory boundaries. By adhering to established data protection principles and providing nondiagnostic decision assistance, the system performs responsibly within the project scope.